



Generative AI

Project 1

Industrial Training Kit

Batch: 2026 | Powered by DecodeLabs

WELCOME TO THE TEAM!

Step into the role of an AI Engineer at DecodeLabs. Project 1 is your essential start: Custom AI Chatbot with Memory. This track isn't about "just calling a basic API"—it's about Stateful Architecture. Before you build complex autonomous agents or RAG applications, you must master the foundational mechanics of maintaining an active in-memory array to store conversation history, dynamically appending every new interaction to the payload to preserve context. By completing this milestone, you are proving you can transform a stateless LLM into a fully contextual, live conversational terminal through pure session state logic. Let's engineer a chatbot that remembers with absolute accuracy.



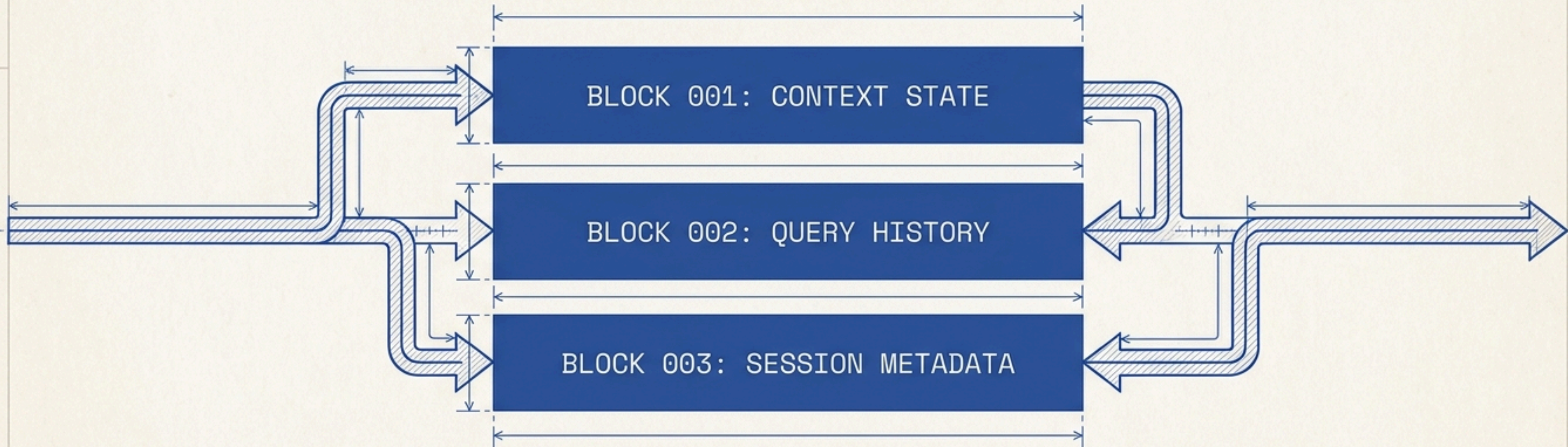
Project 1: Custom AI Chatbot with Memory



- Goal: Build a conversational terminal or web app that remembers previous user messages during a live session.
- Key Requirements:
- Connect to a frontier LLM using an official SDK API key.
- Maintain an active in-memory list array to store the conversation history.
- Append every new user input and model response to the history payload to preserve context.
- Key Skills: API integration, session state management, chat history mechanics.

ENGINEERING STATEFUL CONVERSATIONAL AGENTS

FROM LOCAL MEMORY LOOPS TO ENTERPRISE-SCALE PERSISTENCE

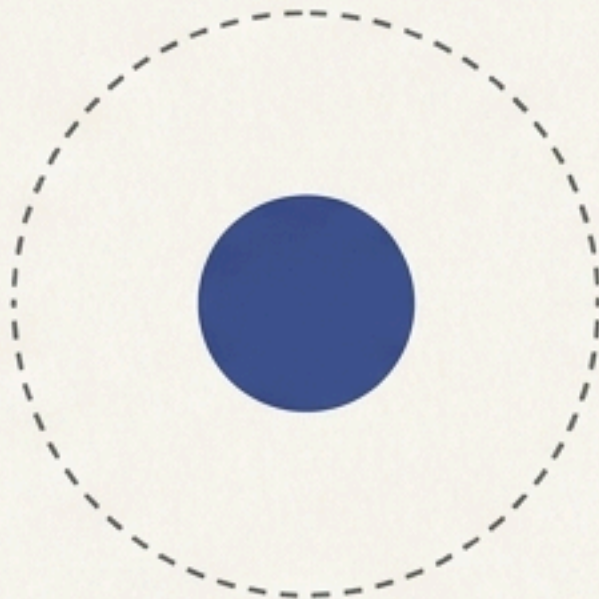


THE AMNESIAC CLOUD

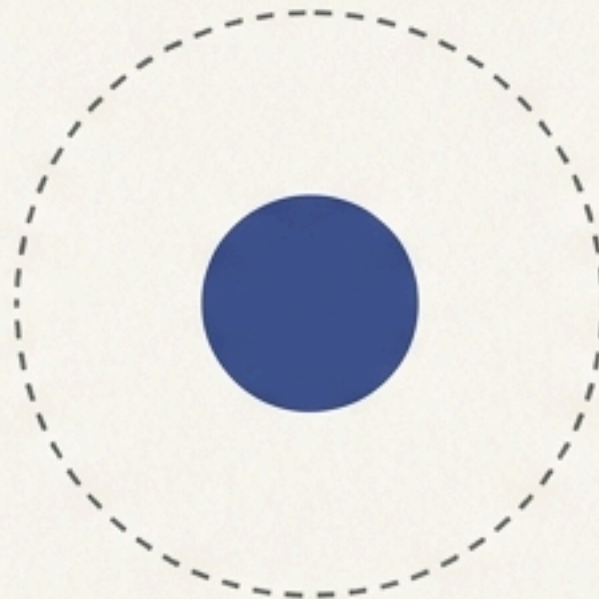
Stateless Cloud Deployment & the Challenge of Continuity

In their native cloud deployment state, Large Language Models (LLMs) operate as entirely stateless completion engines. Every transaction dispatched to a primary model endpoint treats the incoming prompt as a completely isolated transaction.

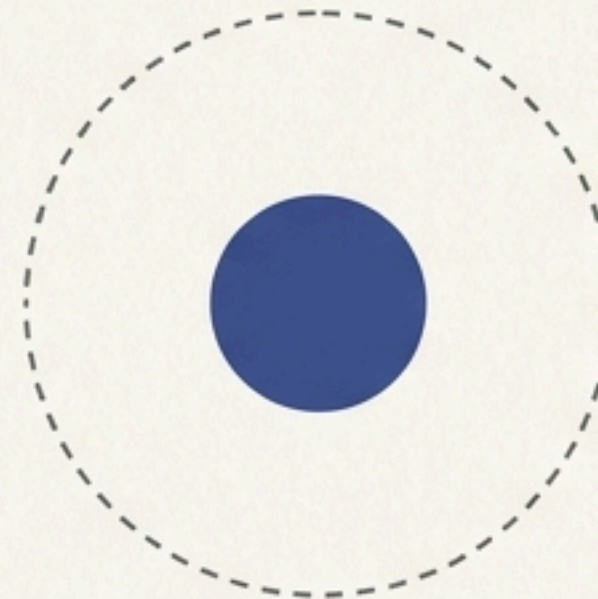
- **INFRASTRUCTURE EFFICIENCY:** Eliminates session tracking and memory overhead at the infrastructure layer.
- **CONVERSATIONAL FAILURE:** Possesses no inherent recollection of prior exchanges, preventing the natural, contextual flow required of interactive companions.



TRANSACTION_01

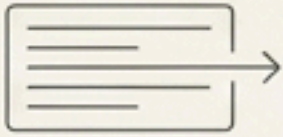
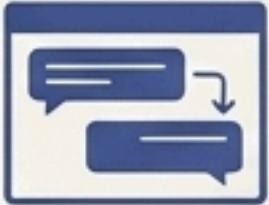
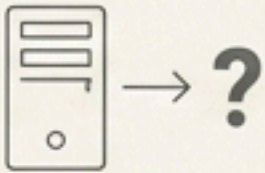
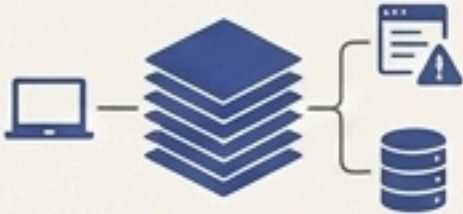
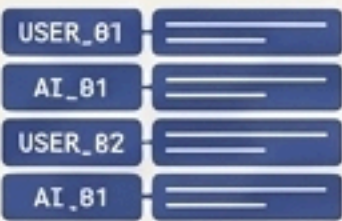


TRANSACTION_02



TRANSACTION_03

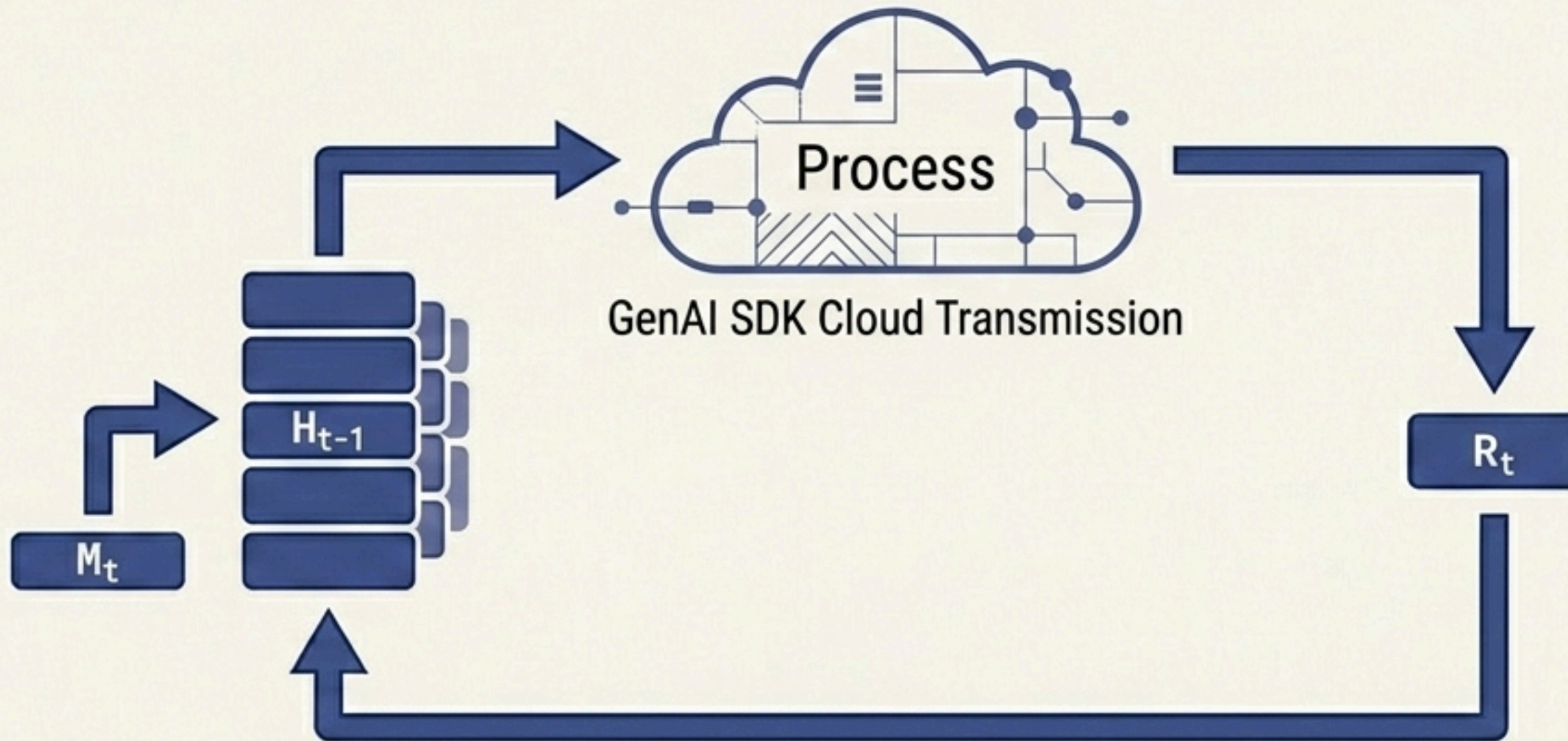
The Paradigm Shift: Stateless vs. Stateful

	Stateless API Completion	Stateful Chat Session
API Endpoint Type	Legacy freeform text completion prompt 	Structured chat completions / active sessions 
State Location	Server treats each request as a total stranger 	Client-side or database-managed historical array 
Context Assembly	Manual text concatenation via custom strings 	Array of structured role-content objects 
Network Overhead	Low for single requests 	High; the entire historical array is sent with every turn 
Downstream Failure Risks	Minimal session-level crashes 	Potential context window overflow or token budget exhaustion 

Constructing the Artificial Memory Loop

Input ($\mathbf{M}_t \cup \mathbf{H}_{t-1}$) $\rightarrow$ Process (GenAI SDK Cloud Transmission) $\rightarrow$ Output ($\mathbf{R}_t$)

- $\mathbf{M}_t$: The current user message at turn t .
- $\mathbf{H}_{t-1}$: The historical array of all message transactions up to turn $t-1$.
- $\mathbf{R}_t$: The generated, context-aware model response.



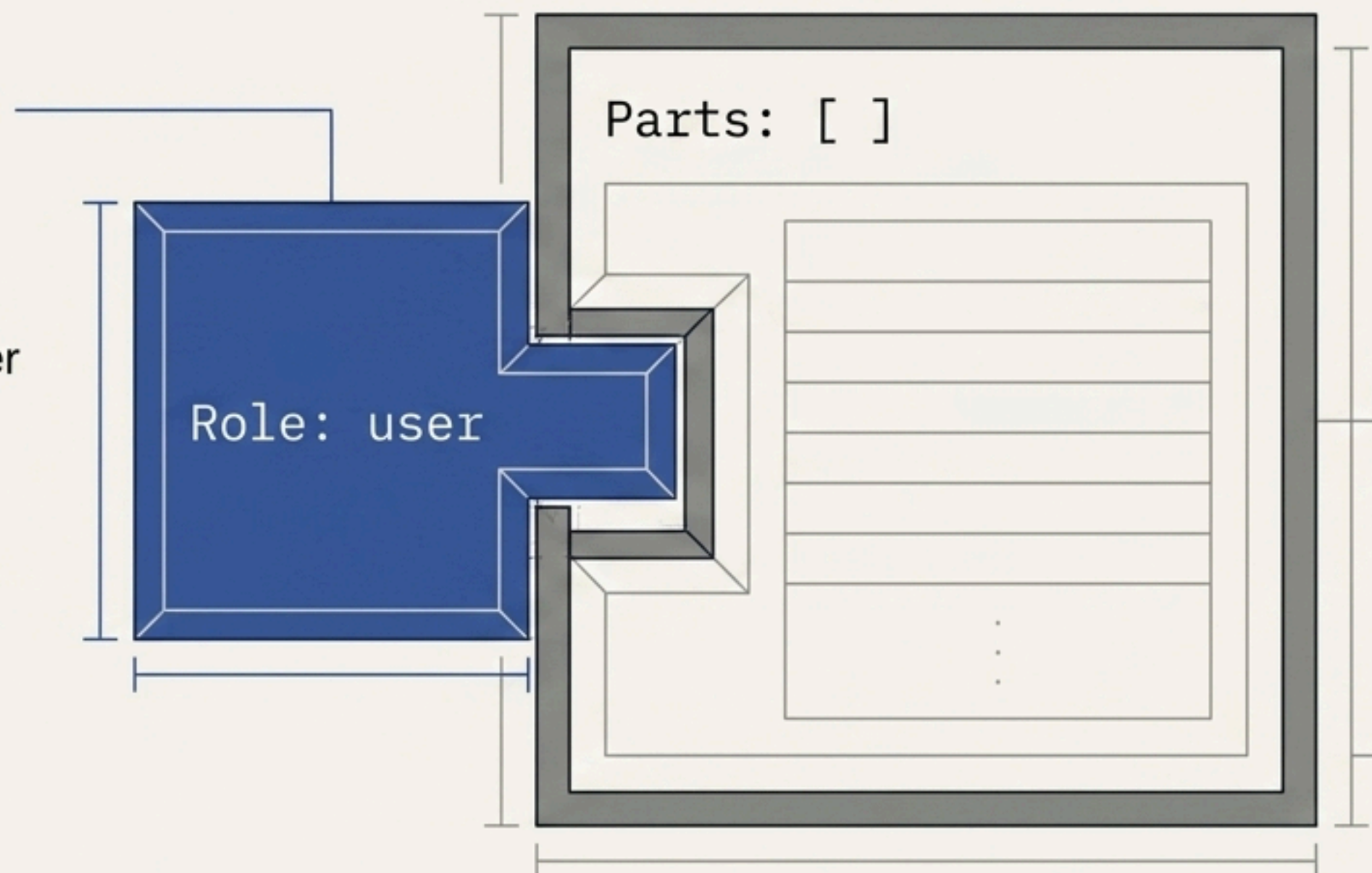
Anatomy of a Chat Session Schema

Modern generative AI SDKs manage conversation data using serialized lists of objects, moving away from legacy raw text prompt concatenation.

Role Parameter:

Categorizes the source of the message.

- **user:** The human conversational partner providing prompts.
- **model:** The generative model providing responses.



Parts Component:

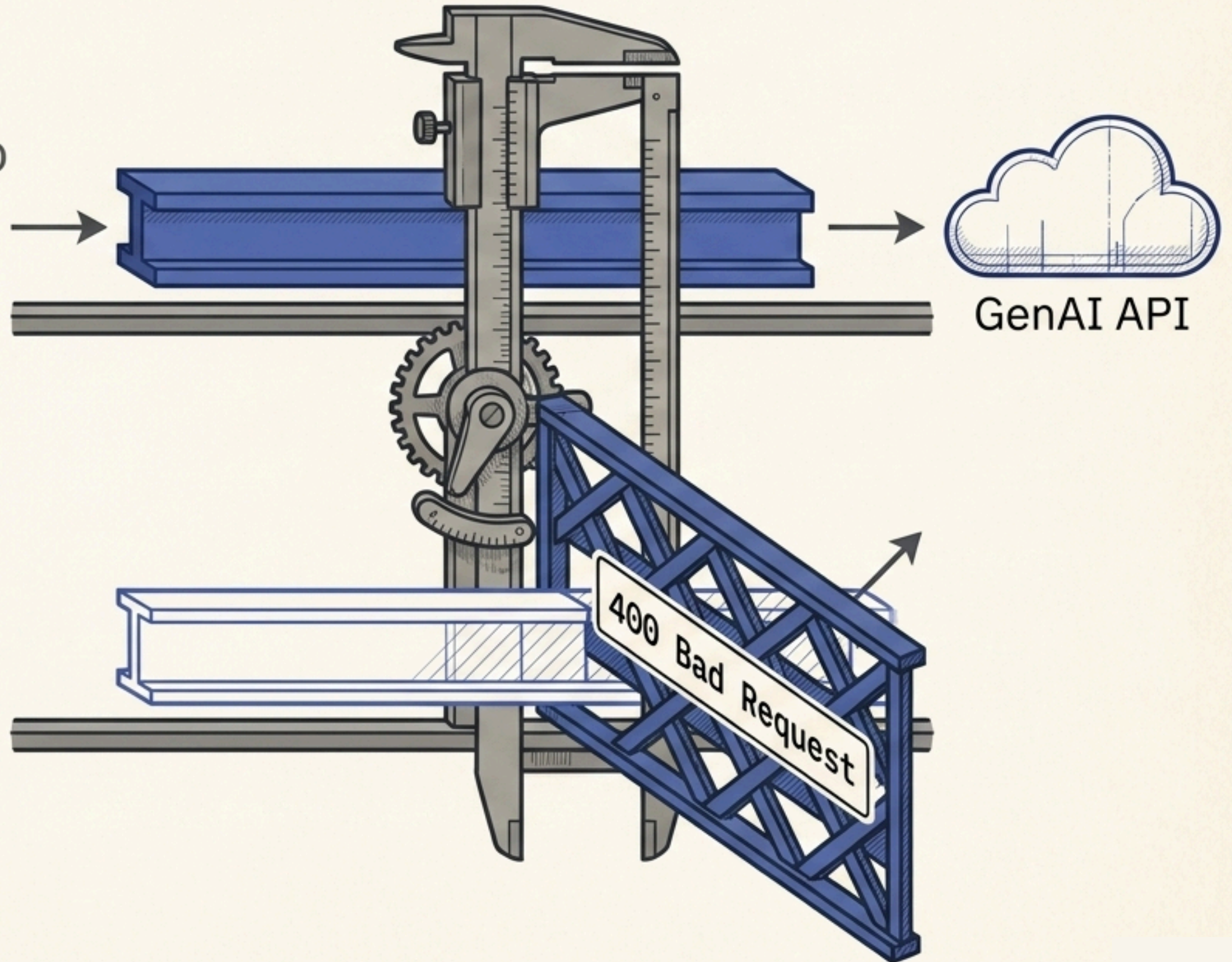
An array housing one or more individual components containing text, multi-modal elements, or tool calls.

The Structural Validation Gate

The Vulnerability: Passing an empty string or whitespace-only payload to an LLM API returns a 400 Bad Request error.

The Impact: Immediately crashes the local terminal script.

The Solution: An explicit conditional guard must evaluate all inputs and block empty transactions before they reach the GenAI API.

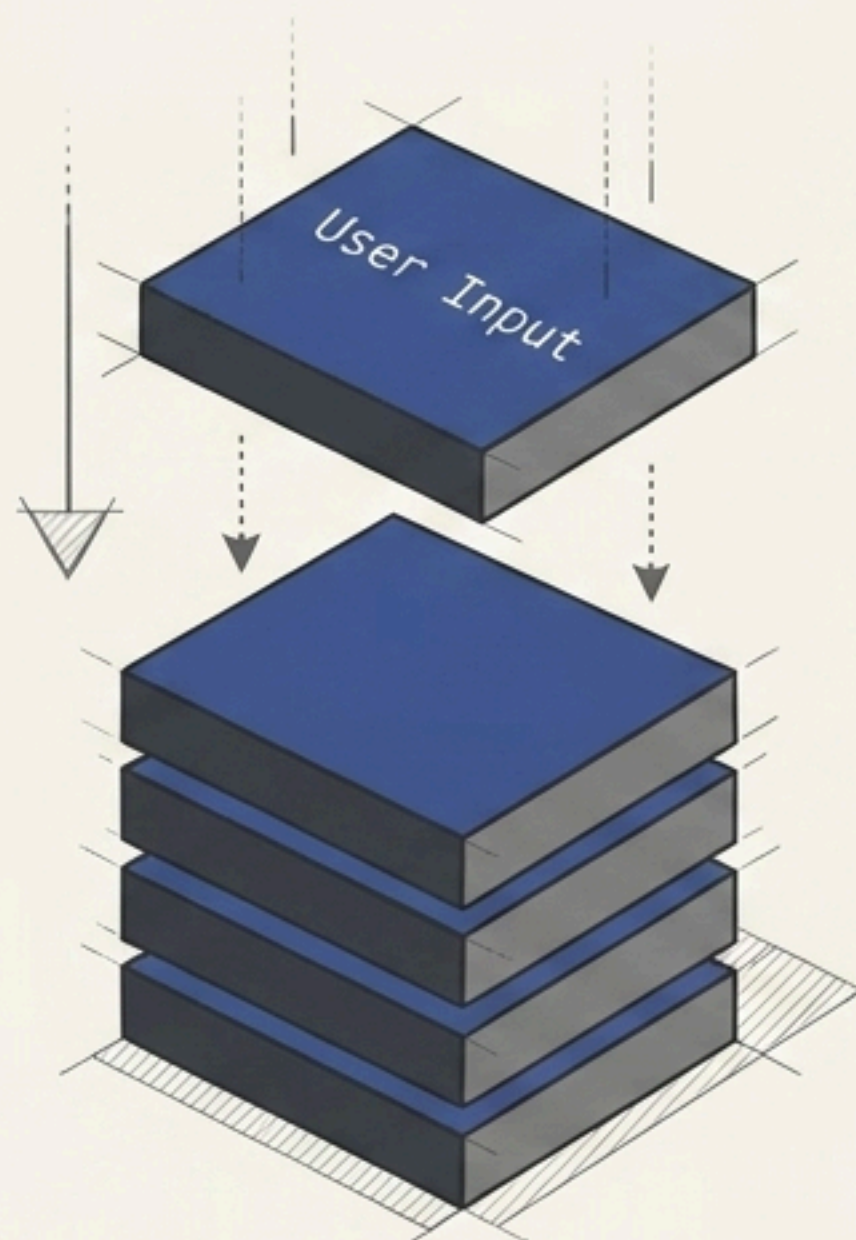


The Terminal Append Sequence

Every conversational turn executes exactly two steps before calling the GenAI API:

1. Ingest & Append:

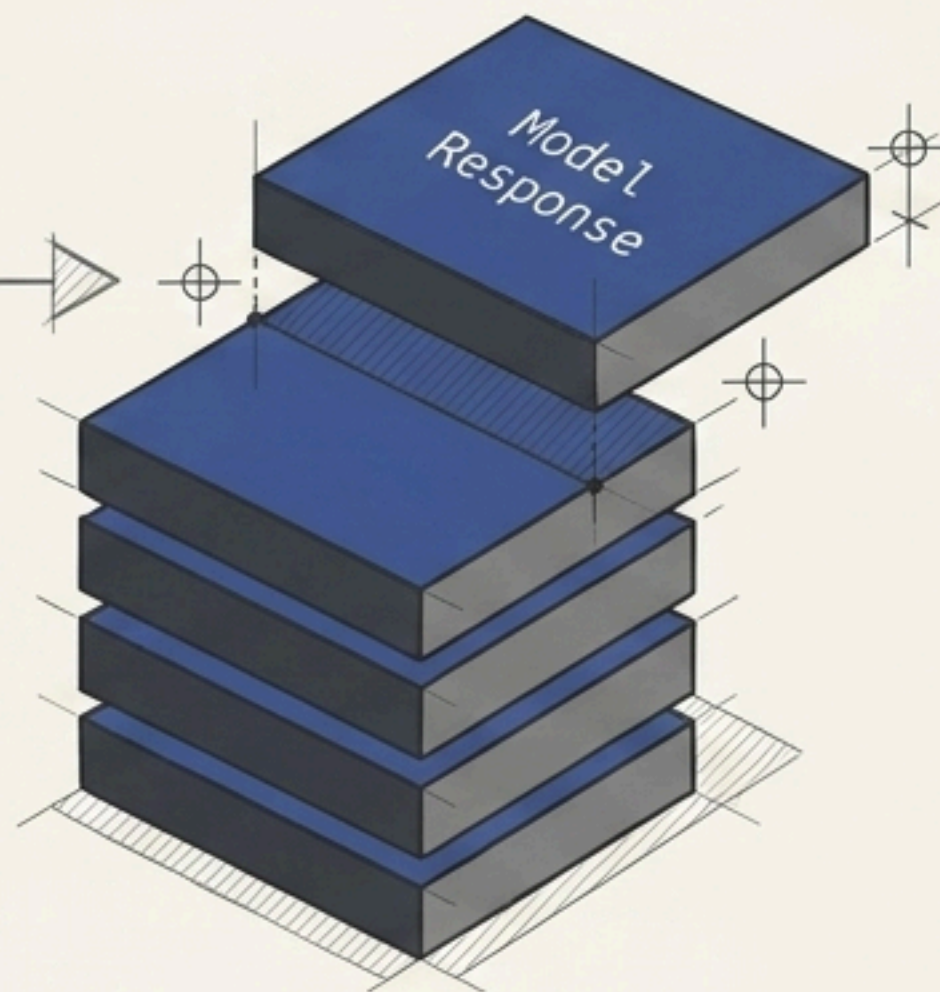
Append the user's validated input to the local history list as a structured user content object.



Transmit

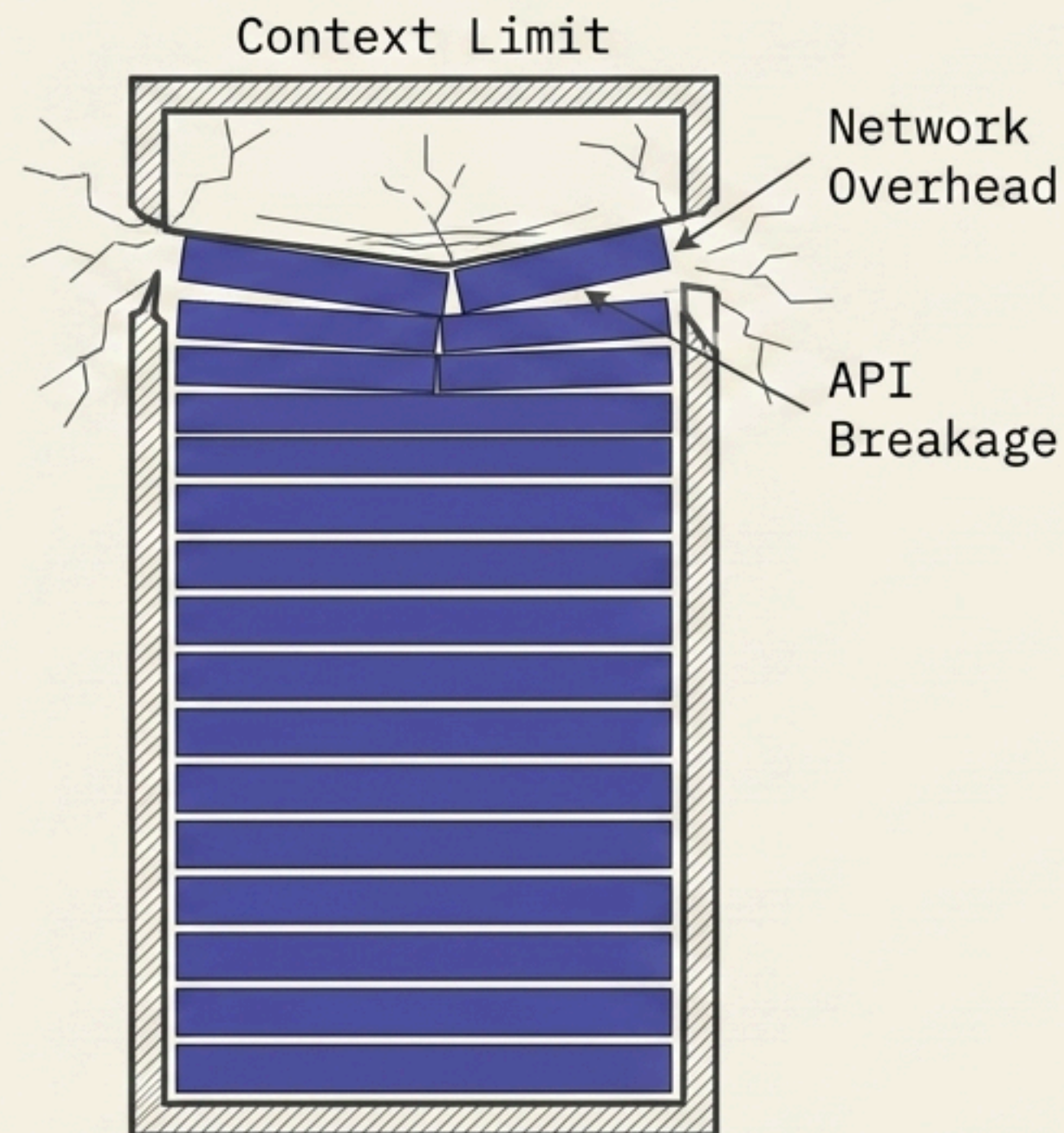
2. Transmit & Record:

Pass the entire updated history list as the payload to the GenAI client, and append the model's generated response to that same list.



The Physical Constraint: Token Exhaustion

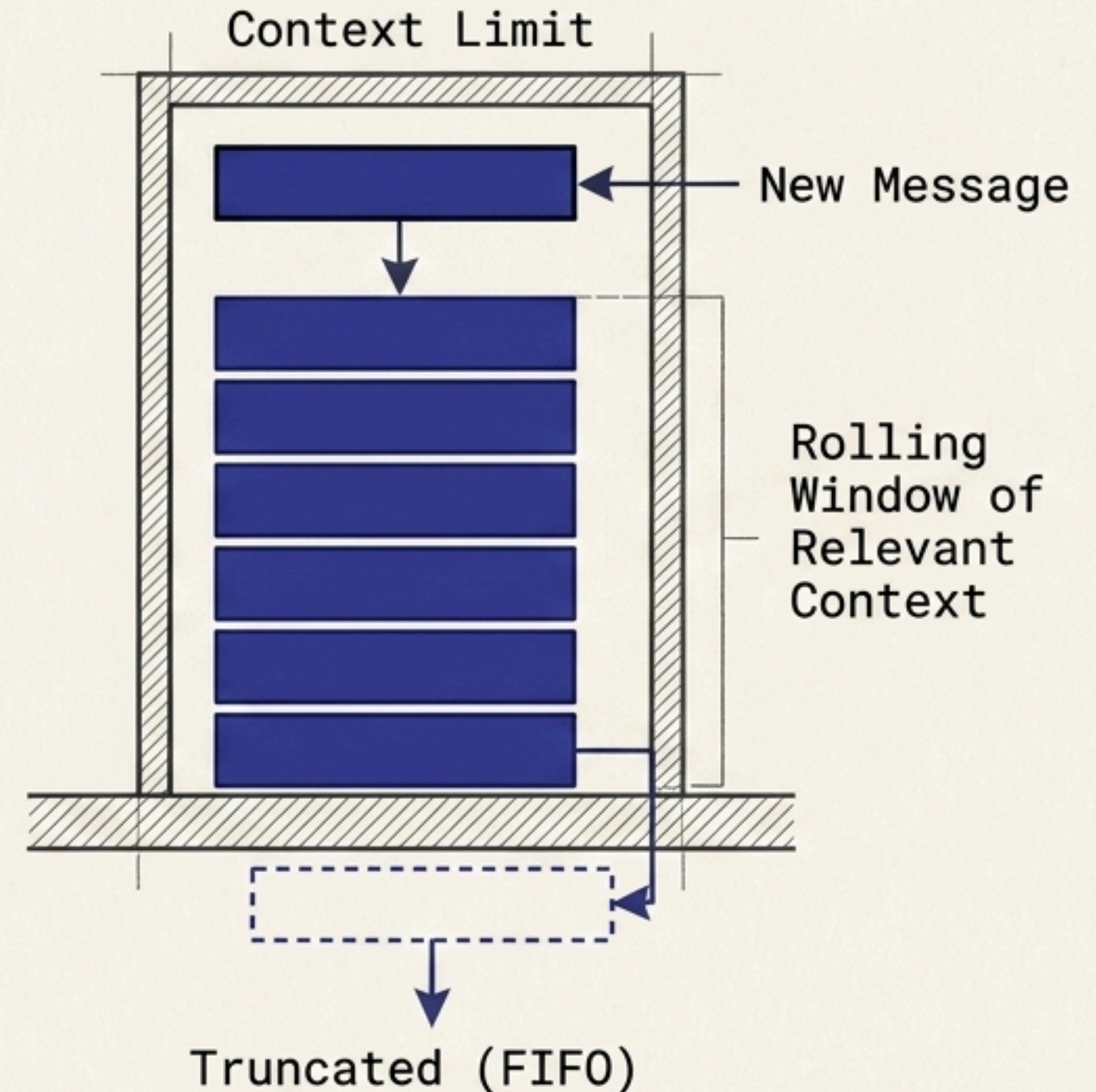
- The Crisis: Models operate with strict physical context windows. Because the stateful loop requires sending the entire historical array with every turn, network overhead scales aggressively.
- The Breaking Point: As the conversation grows, the serialized list of objects approaches the model's token budget limits, resulting in context window overflow and API breakage.



The Sliding Window Algorithm

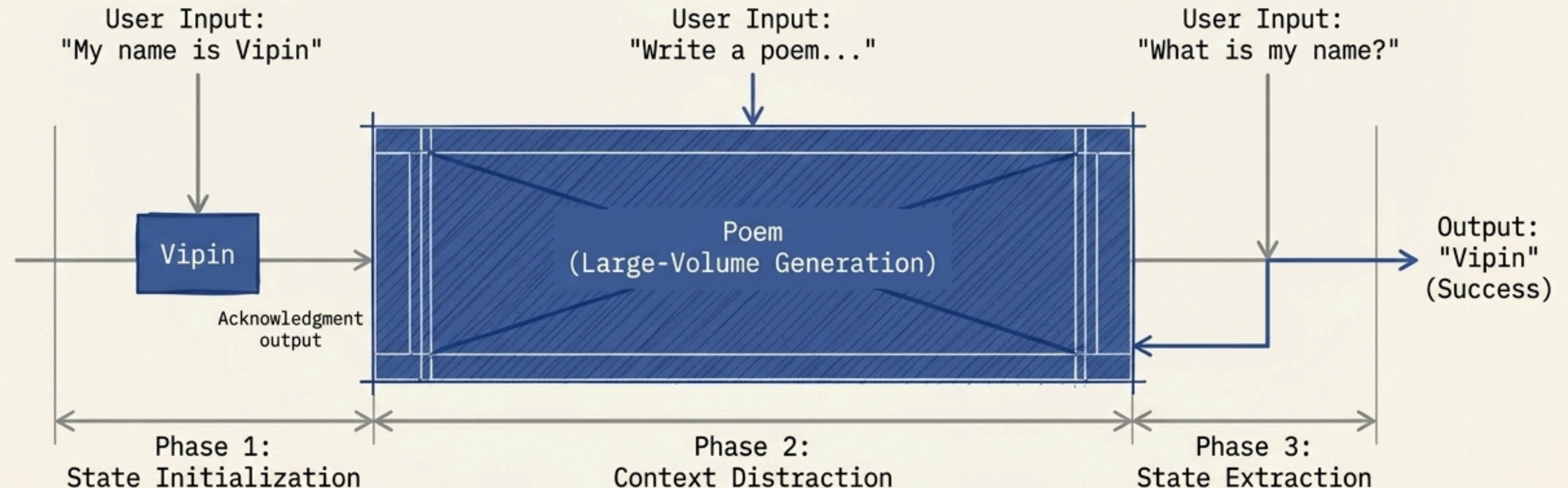
Implementing First-In-First-Out (FIFO) pruning logic to drop the oldest message pairs and safeguard the API.

- The script monitors in-memory array sizes.
- As new turns are appended to the most recent end of the array, the oldest transactions are actively truncated.
- This creates a rolling window of the most relevant recent context.



System Audit: The Memory Exam

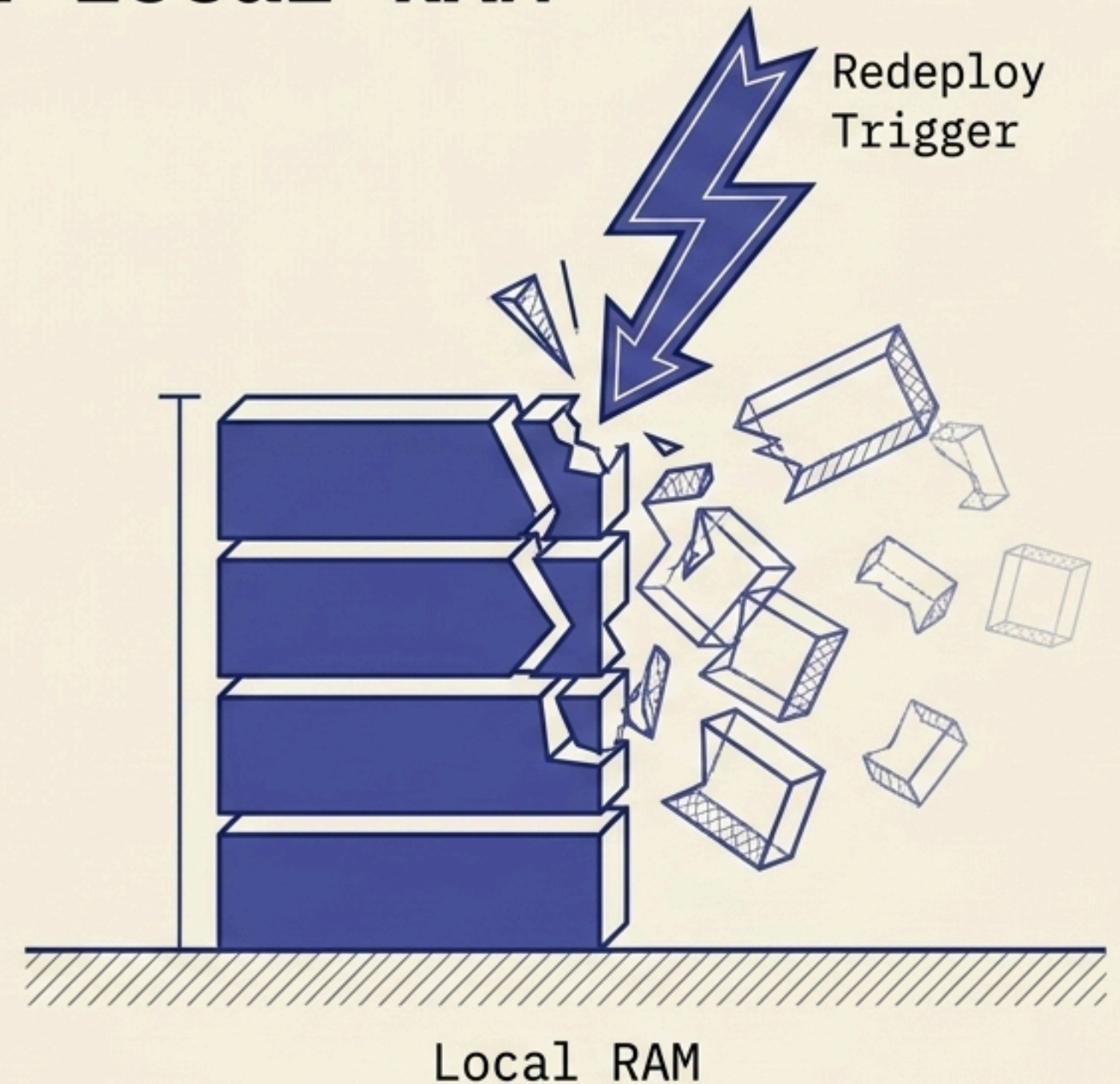
1. **State Initialization:** User inputs "My name is Vipin". Expected: Model generates acknowledgment.
2. **Context Distraction:** User inputs "Write a poem about tech". Expected: Forces a large-volume generation to distract the context window and test large token processing.
3. **State Extraction:** User inputs "What is my name?". Expected: Model successfully extracts and responds with "Vipin" from the sliding window history.



The Ephemeral Nature of Local RAM

Keeping history arrays in local RAM is sufficient for local prototyping, but fatally flawed for SaaS environments.

- **Volatility:** RAM storage is ephemeral. If the host application server is restarted or redeployed, all active session data is permanently lost.
- **Scale:** Enterprise platforms must support millions of concurrent sessions across distributed server clusters.
- **The Requirement:** Conversational state must be isolated from individual application instances and persisted in cloud databases linked to unique session IDs.



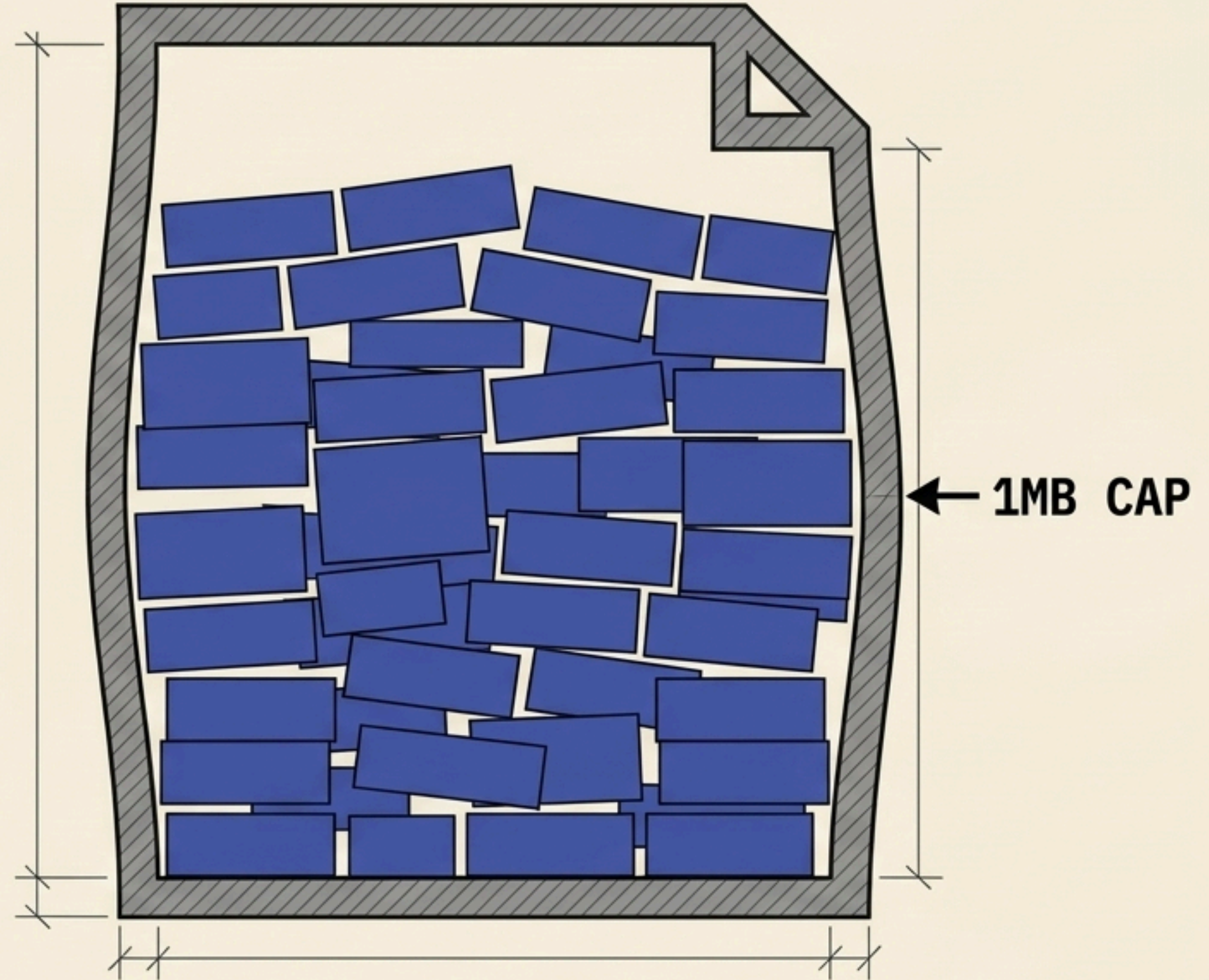
NoSQL Limits: The Cloud Firestore Bottleneck

The Architecture:

A hierarchical structure using a top-level **conversations** collection, with nested **messages** subcollections for individual chat turns.

Scale Constraints:

- **Document Size Limits:** Firestore documents are capped at a strict 1MB size limit. Storing a growing array inside a single parent document will eventually induce a crash.
- **Write/Read Cost Dynamics:** Storing every message as a separate document avoids the 1MB cap, but Firestore charges per individual document read/write. High-frequency chat apps become prohibitively expensive.



Relational Persistence: PostgreSQL

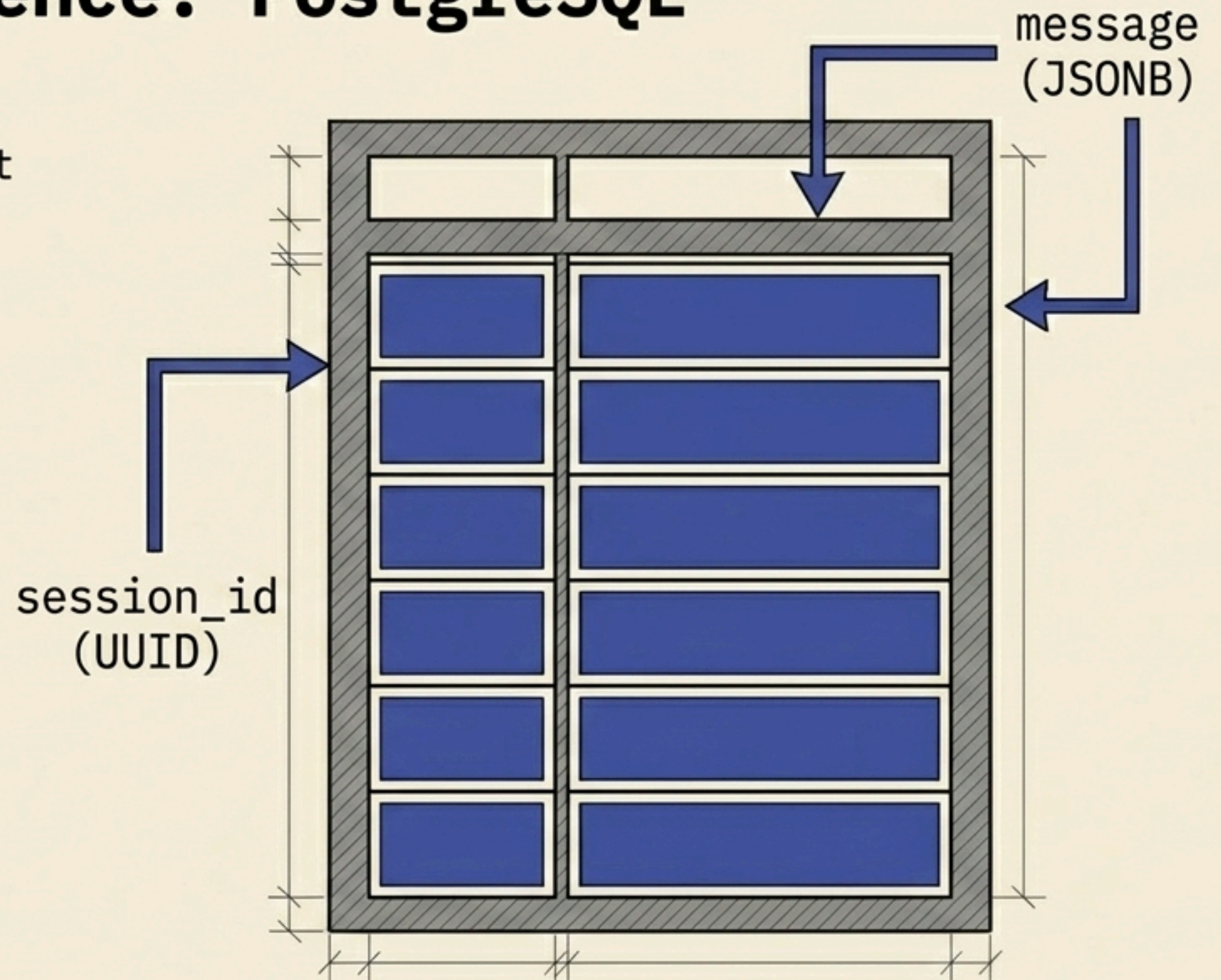
PostgreSQL offers a structured, reliable way to store persistent chat histories using libraries like langchain-postgres.

Implementation Details:

- Data is stored by default in the public database schema.
- The adapter manages connection pooling and creates indexes for fast querying.

Required Schema Columns:

- **session_id:** (UUID) identifies the unique chat.
- **message:** (JSONB) stores the exact content payload and role metadata.



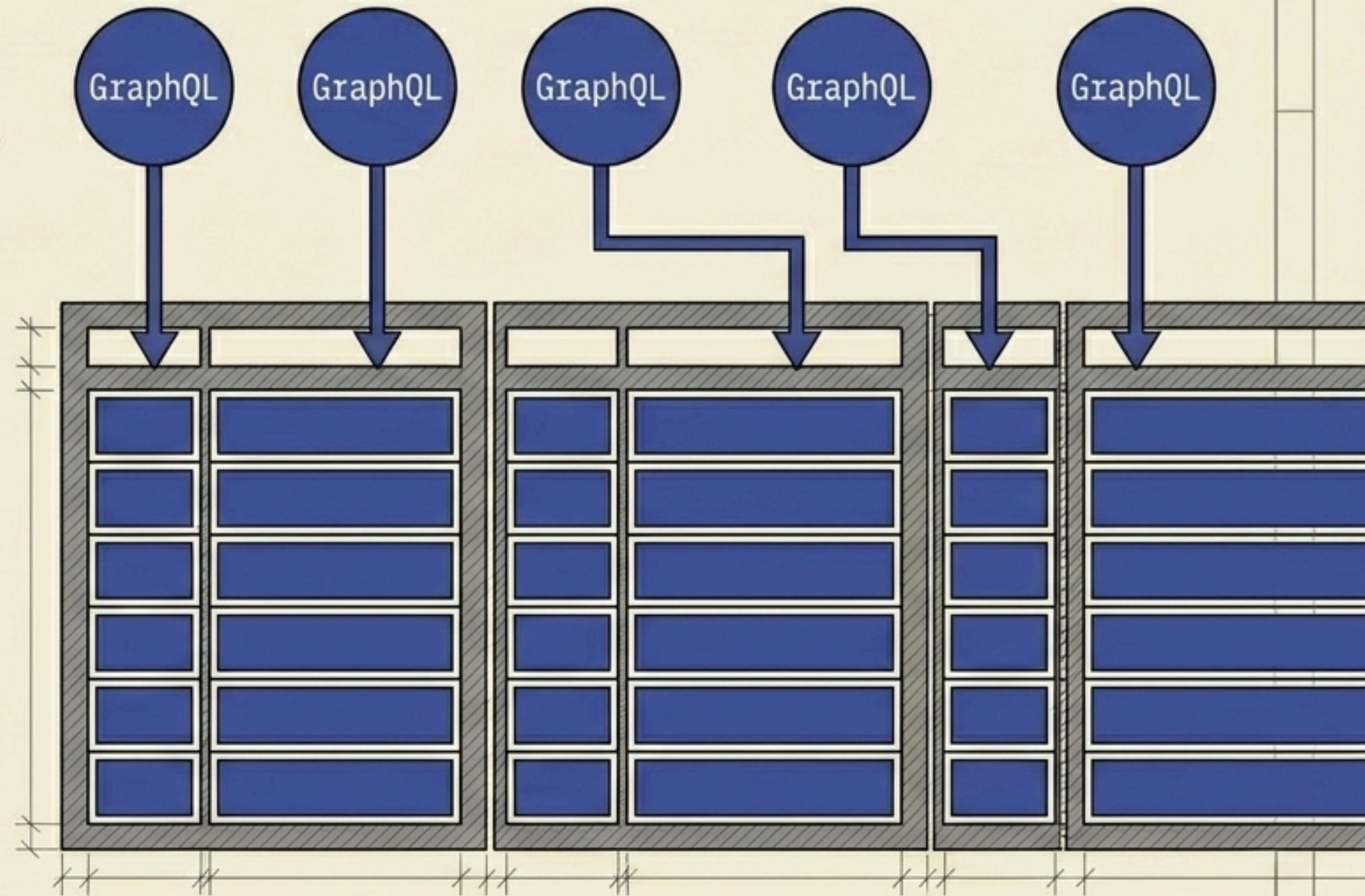
Enterprise Scale: Firebase SQL Connect

The Ultimate Persistence Layer:

Backed by fully managed Cloud SQL for PostgreSQL, addressing both relational structure and horizontal scaling.

Key Features:

- **Automated Infrastructure:** Provides automated database schemas and secure endpoints via GraphQL.
- **Native Postgres Power:** Leverages JSONB columns, vector searches, and granular IAM authentication directly within the Firebase ecosystem.
- **Client Generation:** Automatically generates SDKs for client platforms.



The Enterprise Architecture Matrix

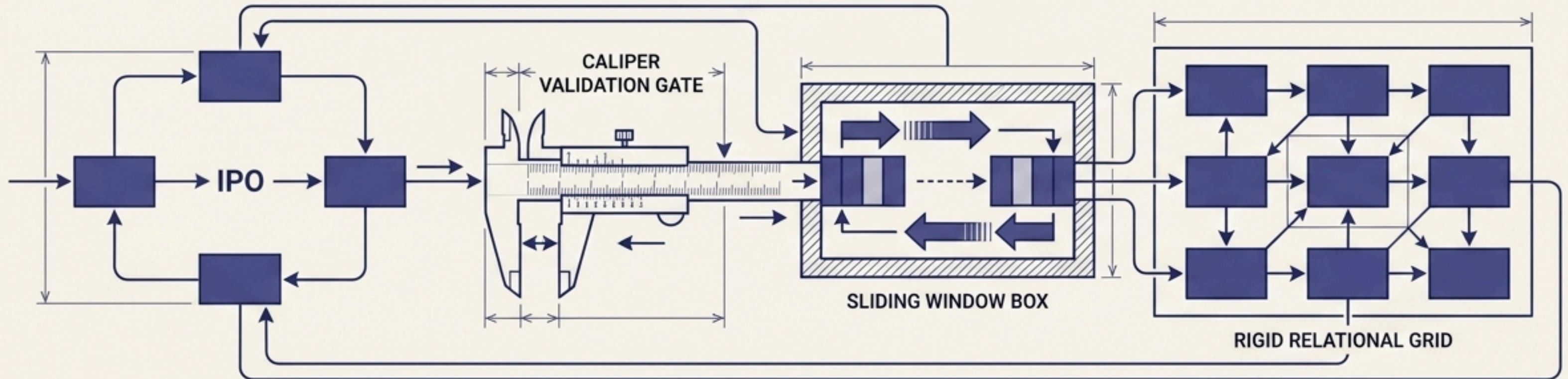
	PostgreSQL	Firestore	SQL Connect
Data Schema Model	Structured tables with JSONB	Schemaless, hierarchical documents	Managed relational schemas via GraphQL
Session Isolation	Primary/foreign keys	Unique document identifiers	SQL row-level security / GraphQL filters
Scale Constraints	Disk space / connection pools	Strict 1MB document size limit	Scaled horizontally via managed Cloud SQL
Billing Model	Flat rate allocated servers	Pay-per-use read/write ops	Tiered pricing based on Cloud SQL usage

Engineering Competency Mapping

Constructing this stateful conversational system establishes deep proficiency across core domains of generative AI architecture:

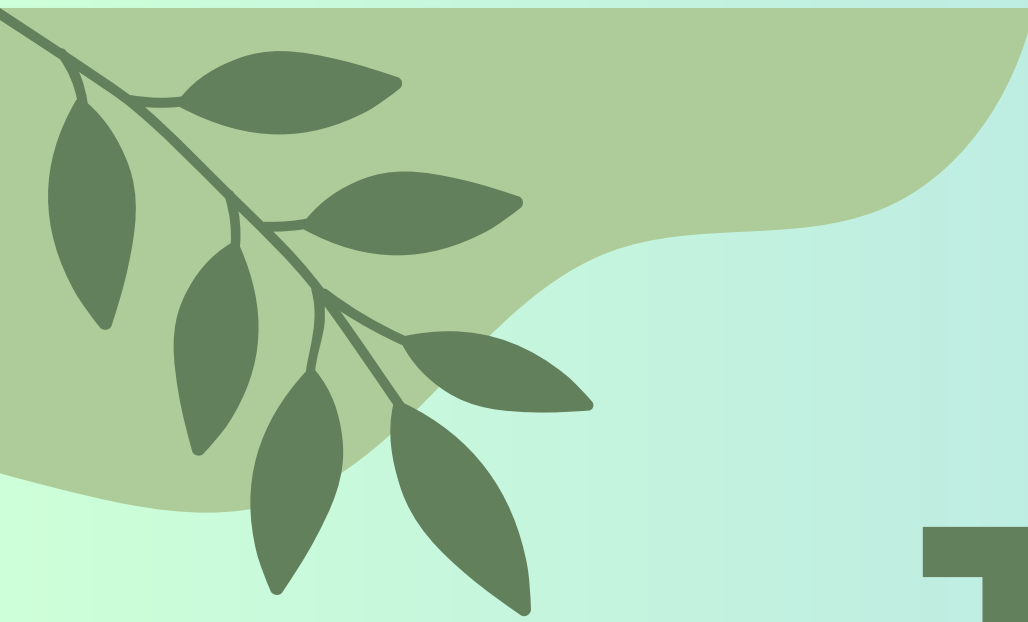
- **Stateless vs. Stateful APIs:** Practical understanding of constructing external memory loops on top of stateless REST endpoints.
- **Session State Management:** Mastery of input validation guards, FIFO truncation, and sliding-window managers to protect application stability.
- **Generative SDK Architectures:** Command over structured payloads and role-content objects, critical for multi-turn conversations and tool calling.

The Completed Blueprint



CONCLUSION

The absolute best way to master Generative AI Engineering is through hands-on system building, not just theory. Don't just aim to complete these projects; take them one by one, experiment with unique solutions—like adding a logic loop to prune older messages when the history exceeds the model's token limit, or designing a clean terminal interface—and treat every out-of-context response or API payload error as a valuable learning opportunity. As you build these milestones at DecodeLabs, you are creating a real-world portfolio that showcases your AI integration and production-level orchestration skills to future employers. Your journey to becoming a professional AI Engineer begins right here, right now, with the very first conversational array you loop today.



THANK YOU

 +91 9236011887

 decodelabs.tech@gmail.com

 www.decodelabs.tech

 GREATER LUCKNOW, INDIA

